

This homework has 10 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. Before you start; before you submit.

If applicable, state the name(s) and username(s) of your collaborator(s).

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

2. Keeping things fresh with potpourri.

- (3 pts) (a) Consider the following algorithm STOOGESORT. Give, with brief justification, a recurrence for the running time $T(n)$ as a function of the input array size n . Then, use the Master Theorem to derive a closed-form solution for the running time. You may ignore off-by-one errors introduced by the ceiling $\lceil 2n/3 \rceil$ since it won't affect the asymptotic running time. *Ungraded Note:* It turns out STOOGESORT is a correct sorting algorithm! Maybe you can figure out why.

```

1: function STOOGESORT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then
3:     return  $A$ 
4:   if  $n = 2$  and  $A[1] > A[n]$  then
5:     swap  $A[1]$  and  $A[n]$ 
6:   if  $n > 2$  then
7:      $t \leftarrow \lceil 2n/3 \rceil$ 
8:     STOOGESORT( $A[1, \dots, t]$ )
9:     STOOGESORT( $A[n - t + 1, \dots, n]$ )
10:    STOOGESORT( $A[1, \dots, t]$ )
11:  return  $A$ 

```

- (3 pts) (b) Consider the following function, which takes as input non-negative integers a and b that are powers of two.

```

1: function ALG( $a, b$ )
2:   if  $a = 1$  or  $b = 1$  or  $a = b$  then return 0
3:   if  $a > b$  then return ALG( $a/2, 2b$ )
4:   if  $b > a$  then return ALG( $2a, b/2$ )

```

Either find a valid potential function to prove that ALG halts on all valid inputs a, b , or show that no such function exists by giving an input on which ALG runs forever with brief justification.

- (3 pts) (c) State, with brief justification, if this algorithm is “efficient” with respect to the input size. You may assume that the divisibility check takes polynomial time with respect to the input size (e.g., using the grade-school division algorithm).

```

1: function ISPRIME( $k$ ) //  $k \geq 2$  is a positive integer
2:   for  $i = 2, \dots, \lfloor \sqrt{k} \rfloor$  do
3:     if  $k$  is divisible by  $i$  then return false
4:   return true

```

3. DFA warmup.

Consider the DFA in Figure 1 below.

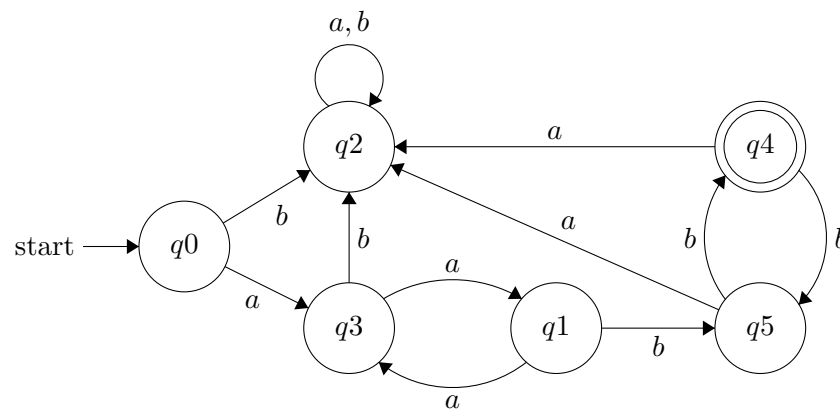


Figure 1: DFA for Question 3

- (5 pts) (a) Recall that a DFA is formally defined as a 5-tuple $(\Sigma, Q, q_0, F, \delta)$. Give these five components for this DFA (Figure 1), with the state-transition function presented in table format, as in lecture.
- (3 pts) (b) Run the DFA on the following input strings, and list all the ones it accepts. (No justification is needed.)
- ε
 - ba
 - $aabb$
 - $aabbbb$
 - $aababba$
 - $aaaabb$
 - $aaabbbb$
- (4 pts) (c) Give, with brief justification, the language that this DFA decides, expressed in ‘string notation’ (alphabet characters; ‘power’, star, and plus superscripts; and the $|$ operator).

4. Constructing DFAs.

For each of the following languages, draw a DFA that decides it. Briefly justify your answers. Use no more than 8 states.

If you're writing your solution in L^AT_EX, you might want to check out [this tool](#) to generate `tikzpicture` script for your DFA.

- (4 pts) (a) The language over alphabet $\{a, b, c\}$ corresponding to $a^+(b|c)^*$.
- (7 pts) (b) The language of binary strings in which the number of 0s mod 3 is 1 *and* the number of 1s mod 2 is 1.
For example, the strings 01, 01000, and 1110 are in this language, while ε , 1, 001, 0011, and 000111 are not.

5. Turing machines.

- (7 pts) (a) Go to <https://turingmachine.io/> and step through the example code titled “binary increment,” which adds 1 to an input integer (represented as a binary string). Modify this machine to add 2 instead of 1 (scroll down on the page for instructions on how to write the code). Give a picture or screenshot of your new machine as your solution. You do not need to provide justification.

Note: The TM model from lecture and TM model used by turingmachine.io are slightly different. The TM model used in class has a one-way infinite tape and accept/reject states. On the other hand, the TM model using by turingmachine.io has a two-way infinite tape and no explicit accept/reject states; instead, the output is on the tape when the ‘done’ state is reached.

- (10 pts) (b) Let L be the language $\{0^{2k}1^k : k \geq 0\}$ over the alphabet $\{0, 1\}$.
Draw a state-machine diagram for a Turing machine that decides L . This drawing can be done however you like (using <https://turingmachine.io/>, drawn in latex, hand-drawn, etc.), as long as it is readable. No justification is necessary. Your machine should follow the lecture convention of having explicit accept and reject states and halt in one of them on every input. You may use turingmachine.io to construct the diagram, but you should label the two halting states accept and reject.

6. Decidability.

Recall that to prove that a problem is decidable, it suffices to provide pseudocode, and prove its correctness. It is not necessary to explicitly describe a Turing Machine that solves the problem, since any problem solvable by pseudocode can be solved by a Turing Machine, and vice versa.

- (5 pts) (a) Prove any finite language $L = \{\ell_1, \dots, \ell_n\}$ is decidable.
- (6 pts) (b) Let $L = (L_1 \setminus L_2) \cup L_3$, where L_1, L_2 , and L_3 are *decidable* languages. Prove that L is decidable.

Note: $A \setminus B := \{x \in A : x \notin B\}$ denotes the *set difference* between A and B .

7. Pac-Man’s journey.

Pac-Man lives on a two-dimensional grid that extends infinitely in all directions, and its moves are controlled by a mysterious outside force. Movement commands come from the alphabet $\Sigma = \{L, R, U, D\}$, which correspond to moving 1 unit left, right, up, and down, respectively.

Pac-Man starts from home at the origin $(0,0)$, and receives an arbitrary (finite) string of commands $s \in \Sigma^*$ to execute in sequence. However, Pac-Man first wants to know more about where it will end up after following the sequence of commands in s .

- (6 pts) (a) Define an *even position* in the grid as one in which both the horizontal and vertical coordinates are even numbers. Give, with brief justification, a DFA that determines whether Pac-Man would end up at an even position after completing the sequence of commands in s . If so, the DFA should accept s ; otherwise, it should reject s .
- (8 pts) (b) Prove that *no* DFA correctly determines, for an arbitrary command sequence s , whether Pac-Man will end up back at home (at the origin) after completing the commands.
- (4 pts) (c) Give an algorithm using pseudocode that takes as input an arbitrary command sequence s and determines whether Pac-Man will end up back at home after completing the sequence. Justify its correctness, but you do not need to analyze the running time.
- So, while a DFA cannot solve Pac-Man's problem, an algorithm (and hence a Turing machine) can!

8. Decision problems and languages.

Recall that the goal of a *decision problem* is to determine whether a given input “object” has a certain “property”, e.g., determine whether a given integer is prime, or whether a given string is a palindrome. In class, we said that any decision problem is equivalent to a *membership problem* for a corresponding *language*. That is, determining whether a given object has the property is equivalent to determining whether its encoding (as a string) is a member of the language.

For each of the following decision problems,

- (i) Define a reasonable (finite) alphabet Σ
- (ii) Give an encoding (over the alphabet) of a representative input object
- (iii) Analyze the length of the encoding
- (iv) Define a language L for the corresponding decision problem

Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet. As examples, we have provided solutions to the first two parts.

- (0 pts) (a) “Does a given non-negative integer k have 3 as its last digit, when written in base 10?”

Solution:

- (i) A reasonable alphabet consists of the decimal digits, $\Sigma = \{0, \dots, 9\}$.
- (ii) Encoding: for an integer k , write it in base 10 in the usual way, as a string of digits. For example, if k is the integer forty-seven, then $\langle k \rangle = 47$. We stress that this is a *string* of the characters 4 and 7, which *represents* the number forty-seven.
- (iii) The length of this encoding would be the number of digits in the value, which is $\Theta(\log k)$.

- (iv) A corresponding language would be $L_{EndsWith3} = \{\langle k \rangle : k \bmod 10 = 3\}$. It would also be acceptable to copy the phrasing of the decision problem, i.e., $L_{EndsWith3} = \{\langle k \rangle : k \text{ written in base 10 has 3 as the last digit}\}$

Alternate Solution: We could use the binary digits $\{0, 1\}$, or hexadecimal digits (0 through 9 and A through F), or the digits from any other base (including unary), or some entirely different-looking alphabet like $\{\star, \otimes, \perp\}$ (though the latter would not be very human-friendly). No matter the choice of (nonempty, finite) alphabet, there is a way to unambiguously encode (represent) non-negative integers as strings of characters from that alphabet.

Note that it would *not* be valid to include all of the non-negative integers in the alphabet, because an alphabet must be a *finite* set, and there are infinitely many non-negative integers.

- (0 pts) (b) “Is a given array of non-negative integers sorted?”

Solution:

- (i) A reasonable alphabet Σ is the set of ASCII characters, or more selectively, the decimal digits along with some separator characters: $\Sigma = \{0, \dots, 9, [, ,, \}\}$. (Notice that a comma is one of these characters.)

Note that we can’t just use the decimal digits alone if we also want some special symbols to indicate the start/end of the array and to separate the elements.

- (ii) Example encoding: For an array $A = \{a_1, a_2, \dots, a_n\}$, encode each of its elements as in the previous part, and list those encodings with appropriate separators. For example, the array A with entries one, two, three, and four would have $\langle A \rangle = [1, 2, 3, 4]$.

- (iii) For an array $A = \{a_1, a_2, \dots, a_n\}$, as we decided to use base 10, each element a_i can be encoded in a string of length $(1 + \lfloor \log_{10}(a_i) \rfloor) = \Theta(\log a_i)$. The length of encoding of A is $(n + 1) + \ell$, where $(n + 1)$ comes from including $n - 1$ commas in the encoding and two brackets $[,]$ and $\ell = \Theta(\sum_{i=1}^n \log a_i)$ is the total length of the encodings of all the elements. Hence the length of the total encoding is $\Theta(n + \sum_{i=1}^n \log a_i)$.

- (iv) The corresponding language is

$$L_{\text{sorted}} = \{\langle A \rangle : A \text{ is a sorted array of non-negative integers}\}$$

- (4 pts) (c) “Given an array S of integers, are all its elements distinct?”
(4 pts) (d) “Is a given undirected graph complete?”
(4 pts) (e) “Does a given C++ program terminate when run on a given input?”

9. Trophy testing (optional extra credit).

Note: This problem is not related to this week's content, and is instead roughly related to divide-and-conquer.

During the National Championship Celebration at Crisler Center, J.J. McCarthy decided to toss the AFCA Coaches' Trophy to Mike Sainristil. Luckily, Mike caught it (as he does with most footballs that come in his vicinity), but the Michigan athletic department was very concerned, since this trophy is made of Waterford Crystal and valued at more than \$34,130. As a result, it has asked researchers at the University's Materials Science and Engineering department to conduct research on Waterford Crystal.

To test its strength, blocks of Waterford Crystal will be dropped from various heights between 1 and n inches (inclusive), for some n of interest. Waterford Crystal is said to have *least breaking height* (LBH) k if a block of it will break when dropped from a height of k inches or more, but will not break if dropped from any fewer number of inches. If it does not break even when dropped from a height of n inches, we say that the LBH is "more than n ". All blocks have the same LBH; the LBH is a property of the material itself, and doesn't vary from block to block.

Your task is to determine the LBH of Waterford Crystal. Since it's a very expensive material, you are given just two blocks. Fortunately, any two blocks of Waterford Crystal have the same LBH. Unfortunately, once a block breaks, it is unusable for future testing.

It is easy to see that if you had only one block, you would have no choice but to drop the block from 1 inch, then from 2, and so on, until the block breaks (or it doesn't even break from a height of n inches). This is because if you were to skip some height, then you would risk prematurely breaking the block without knowing the exact height from which it would have first broken.

But, with two blocks, you can do better! In this problem you will determine the best way to utilize two blocks to determine the LBH, using the fewest number of drops. Instead of expressing the solution as "if I want to determine the LBH among the $n + 1$ possibilities, I can find it using at most $d = d(n)$ drops," it will be cleaner to express the solution *inversely*, as: "With d drops, I can determine the LBH among $n(d) + 1$ different possibilities," where $n = n(d)$ is a function of d .

(2 EC pts)

- (a) Suppose you are given two blocks of Waterford Crystal and allowed no more than d drops, from heights of your choice (which can depend on the results of previous drops). What is the largest value of $n = n(d)$ for which you can determine the LBH among the $n + 1$ different possibilities? Describe your method clearly and concisely in English, give a recurrence and base case for $n(d)$, and give an exact (not asymptotic) closed-form solution.

(1 EC pt)

- (b) Give a short explanation why your algorithm is *optimal*; i.e., why *any algorithm* that uses 2 blocks and at most d drops cannot be guaranteed to work for a value larger than $n = n(d)$, for the $n(d)$ you gave in the previous part. A few sentences of intuitive but logically valid explanation suffice here, but if you wish, you're welcome to prove this rigorously.